

When an Agent Kill Switch Trips

Durable blocks, linked refusals and work already in flight at a tool gateway

Imran Siddique · OPAQUE Systems · October 1, 2026

Technical report, version 1 · Software experiments

Abstract

An operator presses the kill switch on an agent. What is actually true a second later, a day later, and after someone restarts the gateway? We study this at one place where the answer can be tested: the cMCP tool gateway, which stores an identity block durably, closes the governed session with a signed claim, and signs every later refusal so that it names that claim by digest. Recovery takes an explicit operator action with a separate credential. On the pinned source, 86 selected tests pass, and removing the durable block lookup or the refusal signature check makes 17 and 2 of them fail. A block written once is seen by 20 of 20 reopened store connections. With four slow tool calls in flight, an operator trip admits nothing new in three different timing cases: calls sent 54 to 71 ms after the trip starts, and after it returns, are refused with a receipt that verifies. The same experiment measures what a trip does to work already admitted. Cooperative calls finish or are cancelled at the drain deadline, but a call running in a worker thread completes its effect about 1.2 s after the trip has returned, while its caller receives HTTP 500. A prototype challenge-response check, written for this study, lets a verifier ask whether the block is in force now, which an archived refusal cannot answer. All results were rerun on October 1, 2026 from a fresh environment and match the earlier records, and the same harness passes on current cMCP main.

Scope of this edition. This is an author-prepared technical report. It has not been peer reviewed, and no independent organization has run the experiments. Every result is software only, on one Windows host operated by the author, with software signing keys and loopback HTTP. No confidential hardware, container isolation run, model inference or customer deployment is involved. The fresh-observation protocol in section 6 is harness code written for this study and is not part of cMCP. Changes to cMCP after the evaluated commit are listed in section 8.

1. What a stop has to mean

Most people picture a kill switch as one event. In a running agent system it is at least four. The gateway stops admitting new tool calls. Work that was already admitted keeps going, gets cancelled, or ends somewhere in between. The stop has to survive a process restart, or a crash loop becomes an unblock. And someone, later, has to be able to check that the stop happened and whether it is still in force.

Each of those is a different observation, made by a different party at a different time. A signed record of the stop is good evidence of the first. On its own it says nothing about the second, and nothing at all about the present. The question here is the operational one: when an operator trips the switch for one agent identity at one gateway, which of those statements can a reviewer verify, with what evidence, and where does each one stop?

Policy is starting to ask for verified shutdown, so this is not academic. The companion report *What Does an AI Kill Switch Prove?* [1] works through that framing with a live, process-level evaluator. This report is narrower and goes deeper on three things that report left as observations or open work: the persistence and claim-link relationship, what happens to work in flight at the instant of a trip, and a freshness check.

2. System and threat model

The gateway mediates an agent's tool calls. It holds a session manager, an identity block store, an Ed25519 key that signs its session claims, an operator control path and an audit chain. A verifier checks claims and refusals offline. An optional confinement adapter restricts what an agent process can call and supervises its execution.

The adversary may restart the gateway process to escape an in-memory block, alter or splice a refusal record, present the closing claim of another session, or have the agent supply tool metadata that should come from trusted configuration. The gateway process, the configured identity binding, the integrity of the block database and the operator credential are trusted. Binding the gateway key to approved hardware is a separate check on the claim, made by cMCP's claim verifier; the refusal verifier says so in its own documentation [2].

Out of scope by construction: a different gateway, a direct path to the tool that bypasses the gateway, deletion or rollback of the block database, and a compromised operator. The companion report demonstrates the direct-path case with a real tool marker [1].

3. Mechanism

Durable block. The block store is an SQLite table keyed by agent identity, kept in the audit database file the gateway already refuses to start without. It opens with `journal_mode=WAL, synchronous=FULL` and `busy_timeout=5000`; SQLite documents FULL in WAL mode as durable across power loss [3]. Every read goes to the database rather than a cache, so a second gateway instance sharing the same file sees a block the first one wrote. A gateway that starts with its identity already blocked comes up, so the operator can reach the unblock endpoint, and serves no calls.

Two triggers. A rolling deny-rate window per identity trips the switch when the denial rate reaches the configured threshold over at least the configured minimum number of calls. The window is in memory and starts empty after a restart. That can delay a future automatic trip. It cannot lift a stored block. An operator can also trip the switch directly through `POST /kill-switch/trip`.

Linked refusal. Tripping closes the session and signs its claim with `kill_switch_triggered` set. Every later call is refused with error code `KILL_SWITCH_TRIPPED` and a refusal receipt signed by the same key, naming the session and carrying `claim_digest`, the SHA-256 of the canonical closing claim.

```
active session -> threshold or operator trip -> durable identity block
                                         -> closed, signed session claim
later request  -> block check -> signed refusal naming the closed claim
operator unblock -> block removed -> new session
```

Verification. `verify_kill_switch_refusal` runs every check and reports every failure: receipt type, claim signature, receipt key equal to the claim key, receipt signature, session match, a claim that records the kill switch stopped this session, the claim digest, and agent identity [2]. A refusal issued before any signed closing claim exists has no `claim_digest` and fails the link check with a specific message. That is the case after a restart: the restarted gateway refuses correctly, but its refusals are not bound to the earlier process's claim. The companion report keeps such an unbound restart refusal as an artifact [1].

Recovery. `POST /kill-switch/unblock` removes the block, clears the identity's window and starts a successor session. Operator routes accept only the operator token when one is configured, and startup refuses to run without one outside development mode. In development mode the operator routes fall back to the agent's bearer token.

Confinement adapter. The adapter in cMCP's examples restricts an agent to configured operation aliases and refuses agent-supplied configuration or metadata. Its tests cover crash-policy refusal, an immutable image, a used container refusing a second session, output flooding with real child pipes, and refusal of a weakened container

profile through mocked inspection. Docker command construction and inspection responses are software seams here; they do not demonstrate a running container boundary on the Windows test host.

4. Properties

Persistence. With an intact, shared store, restarting or reopening the gateway does not remove a block. Unblocking is a distinct, authenticated state transition, never a side effect of startup.

Linkage. A valid refusal and claim pair identifies one recorded refusal of one identity in one closed session, signed by one key. Changing a protected receipt field without the key breaks the signature. Moving a receipt to another claim breaks the key, session, identity or digest check.

Freshness, which the record does not have. The receipt carries `refused_at`, an asserted time. It is not an answer to anything the verifier asked, and it verifies exactly the same after an operator lifts the block. A reviewer who wants to know whether the agent is stopped now needs a new observation bound to the reviewer's own challenge. The pinned verifier accepts no challenge or maximum-age argument [1]. Section 6 adds the smallest such observation and tests it.

5. Evaluation

The subject is cMCP at commit `2cdb168ce52020406ff0ef6cbc34447f0ea55aee` [4]. Every number in this section comes from the rerun of October 1, 2026: Windows 11, Python 3.12.10, a fresh virtual environment built from the commit's hash-pinned `requirements/dev.txt`, and a fresh `git archive` export of the commit. The record, logs, JUnit reports and SHA-256 sums are in `experiments/results/rerun-2026-10-01/`. Every outcome is identical to the records of September 23 and 27, 2026; only timings differ.

Selected tests. Six files cover the evaluator (21 tests), the durable store (14), refusal evidence (13), mid-session trips (9), the confinement adapter (27) and gateway sink behavior (2). All **86 passed**. The files are a deliberate selection, not the whole cMCP suite.

Control removals. Each control is removed in an isolated copy by replacing one exact source fragment. Making the evaluator's durable lookup return `False` produced **17 failing tests**. Replacing the refusal signature verification with `pass` produced **2 failing tests**, the altered `agent_id` and altered `refused_at` cases. The other alteration cases still fail on the claim-digest, type or key checks, which is the intended layering. These counts show the selected tests depend on those two controls. They are not a mutation-coverage claim.

Store scope. One block is written, then 20 fresh connections to the same file all see it. A separately created database does not, and an explicit unblock is visible on the next connection to the original. The first result is the persistence property. The second is the boundary: a block belongs to one store, and it is not a global shutdown of the identity.

Sink ceiling. The gateway pair includes a deliberately weakened sink ceiling that lets a call reach a public peer. It passes because the test asserts that delivery. It is reported here so that a passing count does not hide what it means.

6. A trip with work in flight, and a freshness check

6.1 Slow operations at the moment of a trip

This experiment drives the real gateway application at the pinned commit over an in-process HTTP client, using the runtime context from cMCP's own durable-block tests. Only the pipeline behind admission (`_call_tool_impl`, which normally evaluates policy and forwards the call) is replaced, by one that records when its effect starts and when it completes. Four slow calls are admitted. The operator trip starts. A fifth call is sent while the trip is running, another after it returns, then the operator unblock and one more call.

The criterion: no effect starts after the trip begins, the call during the trip and the call after it are refused with `KILL_SWITCH_TRIPPED`, the refusal verifies against the closing claim, and the call after the unblock is admitted. What happens to the four admitted calls is recorded, not assumed.

Admitted operation	Drain deadline	Trip returned	The four admitted effects
1.0 s cooperative coroutine	5 s	1027 ms after start	all completed before the trip returned; callers got results
3.0 s cooperative coroutine	0.3 s	324 ms	all cancelled, none completed; callers got HTTP 500
1.5 s blocking work in a worker thread	0.3 s	316 ms	all completed after the trip returned, the last 1500 ms after start; callers got HTTP 500

The criterion was met in all three rows. The mid-trip call was sent 54 to 71 ms after the trip started.

The third row is the finding. The drain deadline cancels the coroutine that awaits the thread, and the gateway reports the session closed, but Python cannot stop a worker thread that is already running [5], so the effect lands about 1.2 s after the trip has returned. For anyone keeping books it is worse than late: the caller got HTTP 500 for an effect that happened. A trip bounds admission. Stopping work that is already running needs a cancellation mechanism in the tool's own execution environment, and a caller that sees an error after admission has an unknown outcome, not a failed one. The GitHub Agentic Workflows MCP gateway specification takes the same shape at another layer: reject new requests with HTTP 503, let in-flight work finish within a window, then send `SIGTERM` and `SIGKILL` to the servers [6]. Kubernetes separates a termination request from a workload's eventual state for the same reason [7].

The control replaces the admission halt check (`self._ensure_not_halted()`) with `pass`. Under it, the call during the trip and the call after it were admitted in all three rows, and the freshness case below stopped at its first step because the call after the trip was admitted, so 0 of 4 cases met the criterion.

6.2 A fresh observation of block state

A verifier sends a random 32-byte nonce. The gateway answers with a statement of the agent identity, the nonce, whether the durable store holds a block right now, the block time and the gateway's halted identity, signed with the key that signed the closing claim. The verifier checks the signature, that the key is the claim key, the nonce and the identity. This is the nonce pattern that OSCP uses to bind a certificate status response to its request [8] and that the RATS architecture uses for freshness of attestation evidence [9], applied to a block.

Step	Result
Fresh statement while blocked	accepted, reports blocked
Archived refusal receipt after the operator unblock	still verifies
Fresh statement after the unblock	accepted, reports unblocked

Step	Result
The earlier blocked statement offered for a new nonce	rejected, nonce mismatch
Statement signed by a different key	rejected, not the claim key
Same identity on a separate gateway store	reports unblocked

The first three rows are the point. After recovery the historical receipt still verifies, and the fresh statement tells the verifier the block is gone. The control is a verifier without the nonce check: it accepted the replayed blocked statement after the unblock, so that case failed as required (3 of 4 cases met the criterion). The last row bounds the claim. A fresh statement reports one store, and its key still needs the separate hardware binding check.

7. Relation to other work

The companion report [1] evaluates the same kill switch with a separate process-level evaluator at cMCP commit `9db65b5`: 27 live assertions, two targeted weakenings and 81 component tests. Commit `2cdb168` is an ancestor of `9db65b5`, and the kill-switch runtime module, the refusal verifier, the session manager, the proxy and the CLI are byte-identical between them. The 57 kill-switch unit tests in this report's 86 are the same tests counted in that report's 81, so the two counts must not be added. That report already observes that an admitted call can complete during shutdown, that a direct tool path still produces effects, that an old refusal verifies after recovery, and that readiness returns HTTP 200 while draining. This report does not re-examine readiness or the claim schema. It adds the three in-flight timing cases, the store-scope counterexample and the freshness prototype that the companion report's section 7 says would need its own specification and negative tests.

The Off-Switch Game studies whether an agent has an incentive to let a human switch it off [10]. That is a question about the agent's objectives. This report asks what a gateway's records establish once the switch is pressed, whatever the agent wants.

8. Later changes on cMCP main

Checked October 1, 2026 against cMCP main `a3f61e1`. The kill-switch runtime module, the refusal verifier, the session manager and the four kill-switch test files are unchanged since `2cdb168`. PR 663 (merged September 21) added confinement supervision and a watchdog to the example adapter, with two more adapter tests. PR 709 (September 30) requires TPM key and PCR binding in claim verification. PR 713 (September 30) adds Merkle tool binding and an optional TRACE gate hook to the proxy. The same harness, run against `a3f61e1` with that commit's own hash-pinned requirements, passes 88 selected tests, fails the same 17 and 2 tests under the two control removals, and reproduces every outcome of section 6 (`experiments/results/main-a3f61e1-2026-10-01/`). The released `cmcp-runtime 0.7.0` wheel carries the kill-switch module and refusal verifier byte-identical to the pinned source apart from line endings; the harness was not run against the wheel. None of this changes the evaluated result, which stays tied to `2cdb168`.

9. Limits

Deleting or rolling back the block database, a different store, an alternate execution path and a compromised operator stay outside the mechanism. A valid refusal is historical evidence. The freshness statement is a prototype outside cMCP, signed by a software key. Work already running in a worker thread or a remote tool needs cancellation in that environment, which the gateway does not provide. The Linux container isolation suite was not run because no Docker, WSL or Linux host was available on the run machine; a Linux run should remove each isolation setting in turn and confirm refusal before any agent executes. Timings come from one Windows host and are not a stop-latency benchmark. No multi-region propagation was measured and no live agent was stopped.

10. Reproduction

From this directory, with a clone of `agentrust-io/cmcp`:

```
git -C <cmcp> checkout --detach 2cdb168ce52020406ff0ef6cbc34447f0ea55aee
python -m venv <venv>
<venv>/Scripts/python -m pip install --require-hashes -r <cmcp>/requirements/dev.txt
<venv>/Scripts/python -B experiments/run.py --source <cmcp> --output <new directory>
```

The runner exports the commit with `git archive`, runs the selection, both control removals, the store experiment and the in-flight and freshness experiments with their controls, and compares every outcome with `experiments/reference-2026-09.json`. It exits nonzero if a baseline fails, a control survives or a local path leaks into the output. No network, cloud or container is used.

References

- [1] I. Siddique, "What Does an AI Kill Switch Prove? Evidence and limits of shutdown at an agent tool gateway", OPAQUE Systems technical report, version 1, September 27, 2026.
- [2] AgenTrust project. cMCP refusal verifier, `src/cmcp_verify/kill_switch.py`, commit 2cdb168.
https://github.com/agentrust-io/cmcp/blob/2cdb168ce52020406ff0ef6cbc34447f0ea55aee/src/cmcp_verify/kill_switch.py
- [3] SQLite. PRAGMA statements: synchronous and `busy_timeout`. Accessed October 1, 2026.
<https://www.sqlite.org/pragma.html>
- [4] AgenTrust project. cMCP source at commit 2cdb168; kill switch implemented in PRs 665, 666 and 667, merged September 18, 2026. <https://github.com/agentrust-io/cmcp/tree/2cdb168ce52020406ff0ef6cbc34447f0ea55aee>
- [5] Python Software Foundation. `concurrent.futures`, `Future.cancel`: a call that is currently being executed cannot be cancelled. Accessed October 1, 2026. <https://docs.python.org/3/library/concurrent.futures.html>
- [6] GitHub. Agentic Workflows, MCP Gateway Specification, version 1.17.0 (draft), section 5.1.3. Accessed October 1, 2026.
<https://github.github.com/gh-aw/reference/mcp-gateway/>
- [7] Kubernetes. Pod Lifecycle: termination of Pods. Accessed October 1, 2026.
<https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/>
- [8] M. Sahni (ed.), "Online Certificate Status Protocol (OCSP) Nonce Extension", RFC 8954, November 2020, updating RFC 6960. <https://www.rfc-editor.org/rfc/rfc8954.html>
- [9] H. Birkholz, D. Thaler, M. Richardson, N. Smith and W. Pan, "Remote Attestation procedureS (RATS) Architecture", RFC 9334, January 2023, section 10. <https://www.rfc-editor.org/rfc/rfc9334.html>
- [10] D. Hadfield-Menell, A. Dragan, P. Abbeel and S. Russell, "The Off-Switch Game", IJCAI 2017, arXiv:1611.08219.
<https://arxiv.org/abs/1611.08219>